

Experiment - 6.1.1. Incremented Date

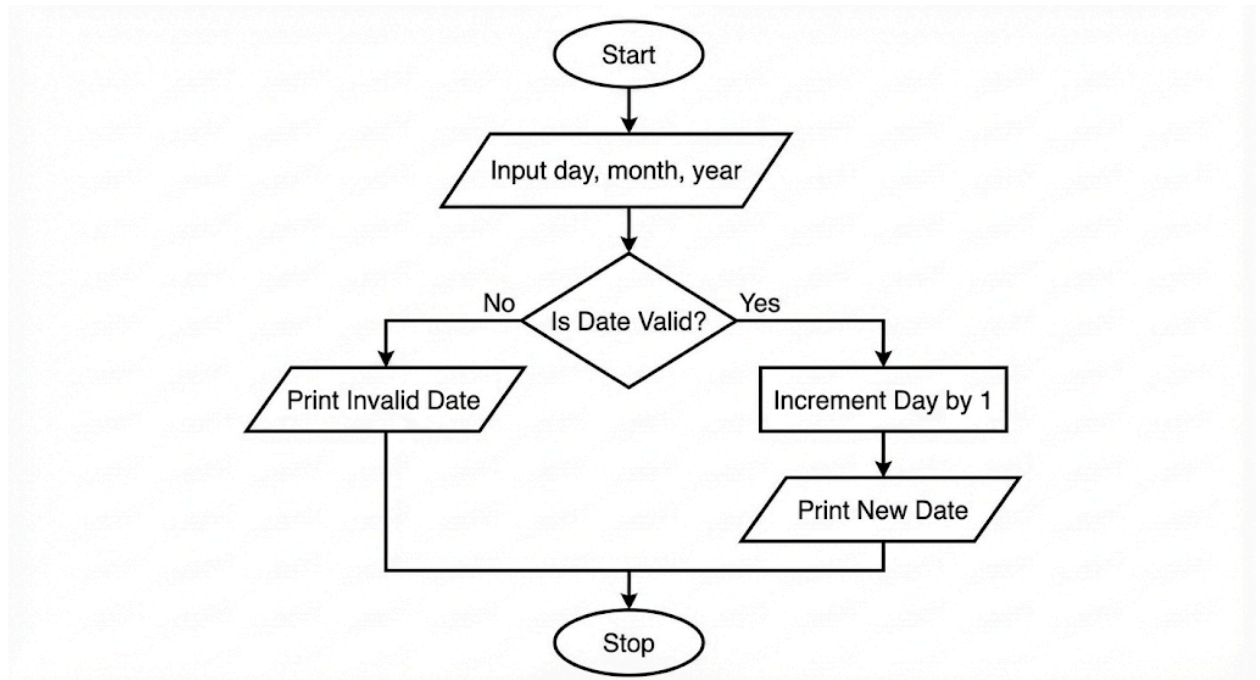
1. Aim

To design and implement a Python program that checks if a given date (day, month, year) is valid according to the Gregorian calendar rules. If the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY format. If the date is invalid, it prints "Invalid Date".

2. Pseudocode

1. **START**
2. **READ** three integers from the user representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True to keep track of date validity.
4. **CHECK** basic validity: **IF** $y \leq 0$ **OR** $m < 1$ **OR** $m > 12$, **SET** flag to False.
5. **IF** flag is True:
 - **DETERMINE** maxdays based on the month:
 - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
 - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
 - **ELSE IF** m is 2 (February):
 - **IF** the year y is a leap year ($(y \% 400 == 0)$ or $(y \% 4 == 0$ and $y \% 100 \neq 0)$): maxdays = 29.
 - **ELSE**: maxdays = 28.
 - **CHECK** day validity: **IF** $d < 1$ **OR** $d > \text{maxdays}$, **SET** flag to False.
6. **IF** flag is True (The date is completely valid):
 - **INCREMENT** the day d by 1.
 - **IF** $d > \text{maxdays}$ (End of the month reached):
 - **SET** d = 1.
 - **INCREMENT** the month m by 1.
 - **IF** $m > 12$ (End of the year reached):
 - **SET** m = 1.
 - **INCREMENT** the year y by 1.
 - **PRINT** the incremented date formatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
 - **PRINT** "Invalid Date".
8. **END**

3. Flowchart



4. Python Program

```

d = int(input())
m = int(input())
y = int(input())

```

```

flag = True
maxdays = 0

```

```

# Check valid year and month bounds
if y <= 0 or m < 1 or m > 12:
    flag = False

```

```

# Determine maximum days in the given month if year and month are valid
if flag:
    if m in (1, 3, 5, 7, 8, 10, 12):
        maxdays = 31
    elif m in (4, 6, 9, 11):
        maxdays = 30
    elif m == 2:
        # Leap year check
        if (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
            maxdays = 29
        else:

```

```
maxdays = 28
```

```
# Check if the day is within the valid range for the month
```

```
if d < 1 or d > maxdays:
```

```
    flag = False
```

```
# Increment date or print invalid message
```

```
if flag:
```

```
    d += 1
```

```
    # Check if we need to roll over to the next month
```

```
    if d > maxdays:
```

```
        d = 1
```

```
        m += 1
```

```
        # Check if we need to roll over to the next year
```

```
        if m > 12:
```

```
            m = 1
```

```
            y += 1
```

```
# Print in zero-padded DD-MM-YYYY format
```

```
print(f"{d:02d}-{m:02d}-{y:04d}")
```

```
else:
```

```
    print("Invalid Date")
```

5. Experiment Screenshot

The screenshot displays the CODETANTRA IDE interface. On the left, a sidebar shows the project structure with a file named 'nextDate.py'. The main editor area contains the following Python code:

```
1 d = int(input())
2 m = int(input())
3 y = int(input())
4
5 flag = True
6
7 if y < 0 or m < 1 or m > 12:
8     flag = False
9
10 if m in (1, 3, 5, 7, 8, 10, 12):
11     maxdays = 31
12 elif m in (4, 6, 9, 11):
13     maxdays = 30
14 elif m == 2:
15     if y % 4 == 0:
16         maxdays = 29
```

Below the code editor, a test runner section shows the results of 5 test cases, all of which passed. The 'Test case 1' section displays the expected output '15-03-2024' and the actual output '15-03-2024'. The bottom of the interface features a terminal and navigation buttons for 'Previous', 'Next', and 'Test cases'.